

Criterion A: Problem Specification

The problem scenario

As someone who has been practicing Biology for my multiple-choice exam that is coming up, I have noticed a lack of resources and quizzes online that allow you to make multiple-choice tests yourself, test your own knowledge, and keep your own score without the need for internet connection and/or creating an account on some form of platform. After talking with my friends and teachers about it, they have noticed a similar issue, and have expressed that having a simple offline desktop application that allows one to create multiple-choice questions to test either their own or someone else's knowledge would be extremely beneficial. As I identified that such an idea could help parents, students, teachers, and many other people, I have decided to form such an application, that could allow them to create their own questions with multiple correct and incorrect answers, with the ability to also attach images to questions that might need it.

The computational context

For this software, I decided to go with the Java programming language, due to its object-oriented nature, which would allow me to easily store questions, data and answers, along with its powerful yet simple syntax, and its platform independence. It also features Swing in its standard library, which will allow me to easily make the GUI for my application.

To store all of the data about the user's progress, I will be using text files, which Java can also read and write to using its standard library, and to allow for the creation of custom multiple-choice questions, I will use YAML files with an external library called Jackson, that will allow me to easily deserialize questions into my Java objects. These technologies are very simple to use, but provide me with all the tools that I will need in order to write and import the questions into the game itself, which will make it simple to use for anyone interested in the app.

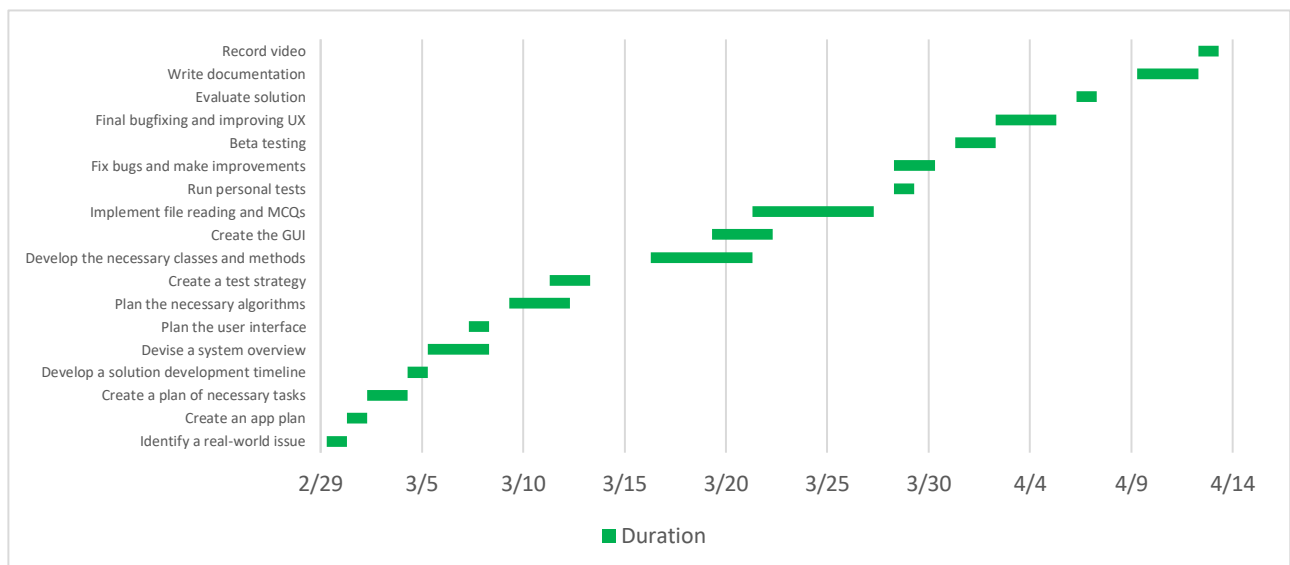
The success criteria

1. Multiple-choice questions with one or more correct answers can be made.
2. Images can, but do not need to, be added to a question.
3. Every question can score a set of points.
4. Points will be saved even after the app has been closed and reopened.
5. The order of answers of a question will be randomized.
6. Upon answering, the app will let the user know if their answer is correct, as well as provide a given explanation/elaboration that is configured together with the question.

Criterion B: Planning

The decomposition and plan

1. Problem specification
 - a. Identify a solution to the real-life problem
 - b. Create an application plan
2. Planning and system overview
 - a. Create a plan of necessary tasks
 - b. Develop a solution development timeline
 - c. Devise a system overview
 - d. Plan the user interface
 - e. Plan the necessary algorithms
 - f. Create a test strategy
3. Development
 - a. Develop the necessary classes and methods
 - b. Create the GUI
 - c. Implement file reading for scores and multiple-choice questions
4. Testing
 - a. Run personal tests according to the testing strategy
 - b. Fix bugs and make improvements
 - c. Beta test the program with a wider audience
 - d. Fix all bugs and improve user experience
5. Documentation
 - a. Evaluate the program
 - b. Write the software documentation
 - c. Record the video showcasing functionality



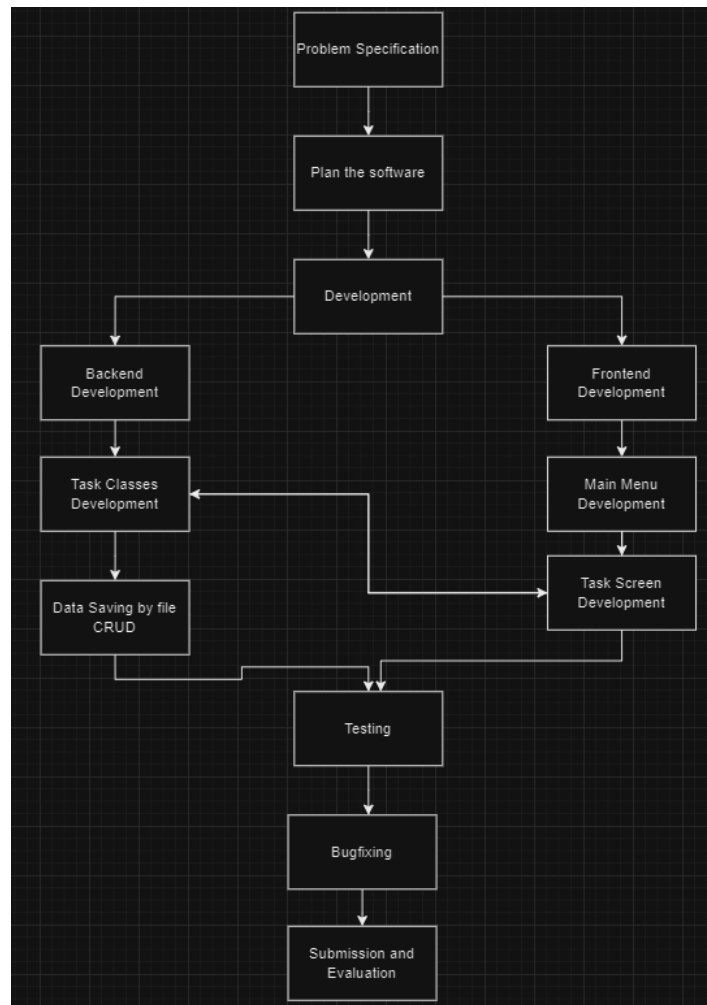


Figure 1: Development plan decomposition diagram

Planned use of libraries:

Library	Usage
Java Swing	Creation of all menus and GUIs through the use of JFrames, JButtons, etc. All click actions will be handled through lambda expressions.
Jackson	Deserialization of Java class objects into file datatypes.
Jackson YAML Datatype	Adding YAML support to Jackson in order to write questions inside of YAML files.

I plan on using Maven for this project in order to add all of the dependencies that I will need for this solution.

Criterion C: System Overview

The system model

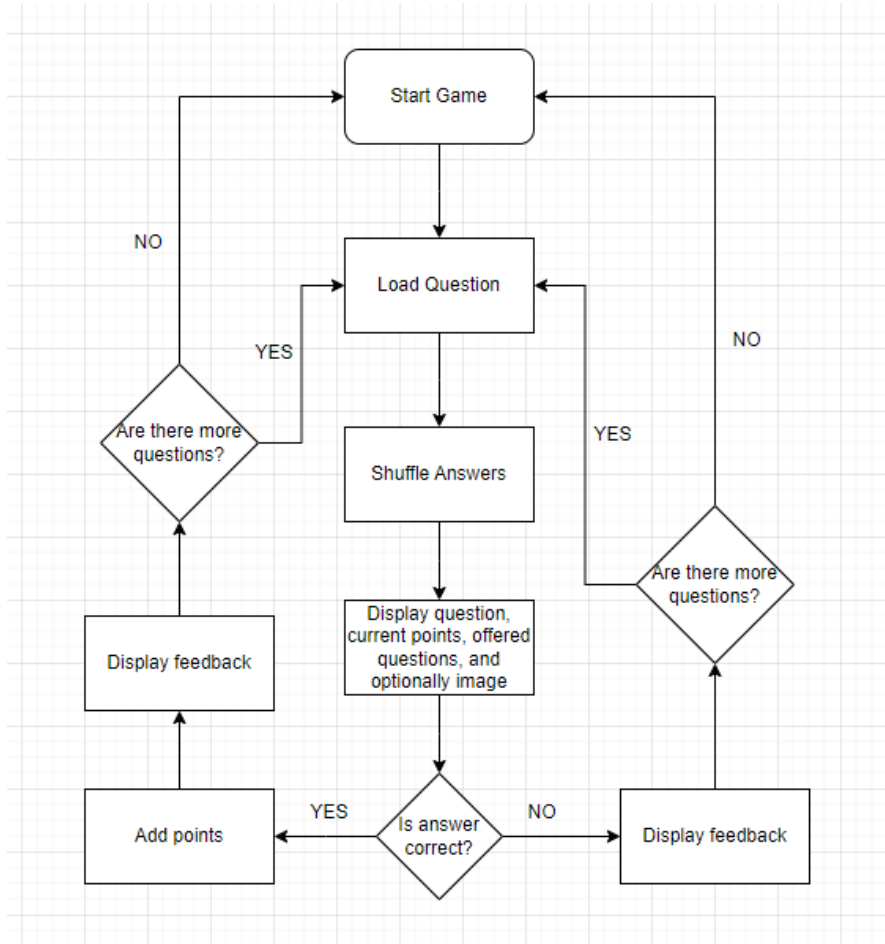


Image 1: The Game Loop

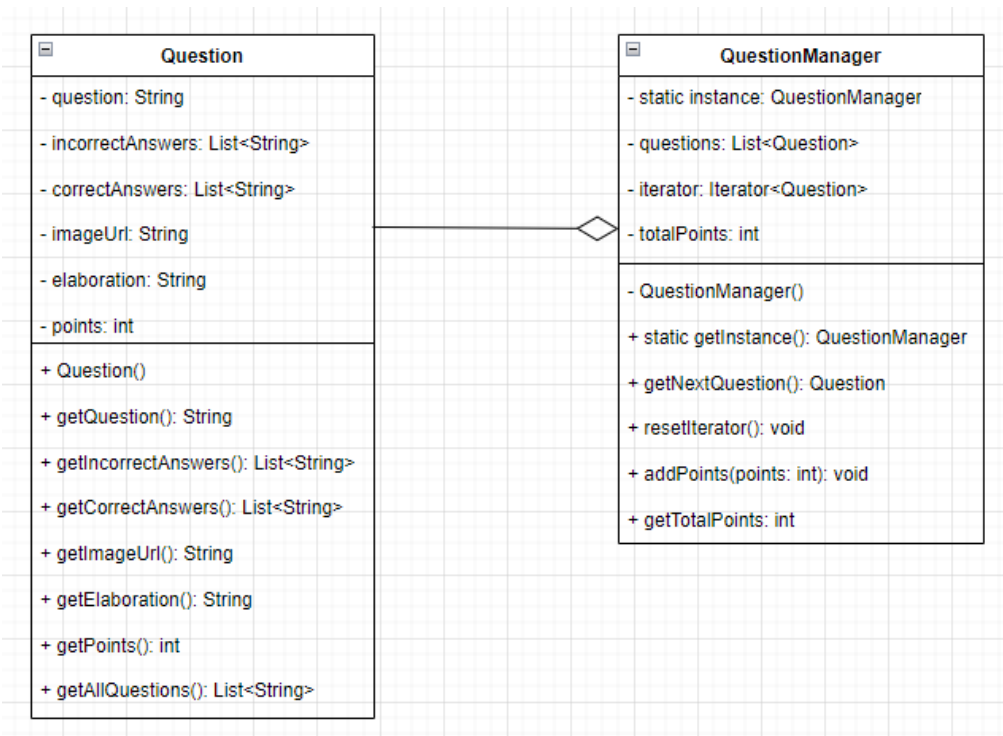


Image 2: The UML Class Diagram Model

questions.yml - Notepad

File Edit Format View Help

```

question: "What is the capital of France?"
correctAnswers: ["Paris"]
incorrectAnswers: ["London", "Berlin", "Madrid"]
imageUrl: ''
elaboration: "Paris is the capital of France."
points: 5
---
question: "What is the tallest montain / mountain top?"
correctAnswers: ["Mount Everest", "The Himalayas"]
incorrectAnswers: ["K2", "Kilimanjaro", "Alpes"]
imageUrl: ''
elaboration: "The tallest mountain in the world are the Himalayas, with its mountain top called Mount Everest."
points: 5
---
question: "What is displayed on the image?"
correctAnswers: ["Cat"]
incorrectAnswers: ["Dog", "Parrot", "Owl"]
imageUrl: 'https://example.com/cat.jpg'
elaboration: "The image shows a cat."
points: 2
  
```

Image 3: The questions file setup configuration

The pseudocode algorithms

Read the contents of the questions.yml file and save them:

```
var questionsFile = open file('questions.yml')
var jackson = new Mapper(YAML())
var stream = questionsFile.inputStream
var mappingIterator = jackson.forClass(Questions.class).read(stream)
questions = mappingIterator.readAll()
```

Read the saved points data file and load the points from previous sessions:

```
method getPastPoints() {
    var file = open file('savedata.txt')
    if not file.exists {
        file.create()
        return 0
    } else {
        var text = file.read()
        try {
            return int.parse(text)
        } catch (Exception e) {
            throw exception
        }
    }
}
```

Display a question:

```

method displayQuestionGui(gui, question, mainMenu) {
    gui.addLabel(question.getQuestion())
    if (question.getImageUrl().isEmptyOrNull())
        gui.addLabel(ImageIO.open(question.getImageUrl()))
    for (var answer in question.getAllAnswers()) {
        var button = create button(answer)
        button.clickAction {
            if (answer is in question.getCorrectAnswers())
                addPoints(question.getPoints())
            displayFeedback(question)
        }
        gui.addButton(button)
    }
    var back = create button('Back to Menu')
    back.clickAction {
        gui.close()
        mainMenu.open()
    }
    gui.addButton(back)
}

```

Get all answers of a question randomized/shuffled:

```

method getAllAnswers() {
    var answers = copy of question.getIncorrectAnswers()
    answers.addAll(question.getCorrectAnswers())
    Collections.shuffle(answers)
    return answers
}

```

The testing strategy

Success criterion	Test	Expected result
1 and 2	1. Add a well-formatted YAML object that can be serialized into the Question class, with a link to an image. 2. Add another such object, but without a link to an image. 3. Add a faulty YAML object.	1. The question will be correctly saved and can be seen in the program with its respective image. 2. The question is correctly saved, and when in the program, the question and answers are shown, with no image. 3. The questions cannot be opened.
3 and 4	1. Answer a question correctly. 2. Answer a question incorrectly. 3. Skip a question. 4. Close the app and reopen it.	1. The amount of points designated for the question is awarded. 2. No points are awarded. 3. No points are awarded. 4. The points obtained before closing the program will be saved after the program is reopened.
5	1. Reopen the same question multiple times.	1. The order of the answers for the question will not be the same when reopening the question.
6	1. Answer a question correctly. 2. Answer a question incorrectly.	1. The program displays a pop-up telling the user that they have correctly answered, will inform them of how many points they've scored and will show the appropriate explanation for the answer. 2. The program displays a pop-up telling the user that they have incorrectly answered and will show the explanation for the correct answer.

Criterion D: Development

Technique 1: File reading

```
try {
    File file = new File("savedata.txt");
    if (file.exists()) {
        Scanner scanner = new Scanner(file);
        totalPoints = scanner.nextInt();
        scanner.close();
    } else {
        totalPoints = 0;
    }
} catch (Exception e) {
    throw new IllegalArgumentException("Invalid points for questions: " +
e.getMessage());
}
```

The following code reads the “savedata.txt” file in order to determine how many points the user had previously, before opening the program again. It stores past points persistently. It is wrapped in a try-catch statement in order to ensure exception handling. A file object is created with the required file location, and then it is checked if the file exists, if so, a scanner is initialized to scan the integer in the file, and closed after that. If the file does not exist, the total amount of points is initialized as 0. The strength of this approach is the simplicity and the fact that it ensures that points are saved (the file writing technique will be explained shortly). The disadvantage of this approach is that it provides no feedback to the user if an exception arises, apart from throwing an exception.

The testing strategy is very effective in testing this algorithm, as testing whether or not points have been saved between sessions will also determine whether this algorithm is functional.

Technique 2: File writing

```
public void addPoints(int points) {
    totalPoints += points;
    try {
        File file = new File("savedata.txt");
        if (!file.exists()) {
            file.createNewFile();
        }
        Formatter formatter = new Formatter(file);
        formatter.format("%d", totalPoints);
        formatter.close();
    } catch (Exception e) {
        throw new IllegalArgumentException("Invalid points for questions: " +
e.getMessage());
    }
}
```

The following method writes the newly added points to the “savedata.txt” file, and adds them to the totalPoints variable. If the file does not exist, it is created, and after that, using a formatter, the totalPoints variable is written to the text of the file, and is closed afterwards. The advantages and disadvantages of this approach remain the same as those outlined for file reading.

Likewise for the algorithm in technique 1, the testing strategy covers and tests this algorithm.

Technique 3: YAML Objects and OOP

```

ObjectMapper mapper = new ObjectMapper(new YAMLFactory());
mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
try (InputStream stream = new FileInputStream("questions.yml")) {
    MappingIterator<Question> questionsIterator =
mapper.readerFor(Question.class).readValues(stream);
    questions = questionsIterator.readAll();
} catch (Exception e) {
    throw new IllegalArgumentException("Invalid questions file: " +
e.getMessage());
}

```

```

class Question {
    private String question;
    private List<String> incorrectAnswers;
    private List<String> correctAnswers;
    private String imageUrl;
    private String elaboration;
    private int points;

    public String getQuestion() {
        return question;
    }

    public List<String> getIncorrectAnswers() {
        return incorrectAnswers;
    }

    public List<String> getCorrectAnswers() {
        return correctAnswers;
    }

    public String getImageUrl() {
        return imageUrl;
    }

    public String getElaboration() {
        return elaboration;
    }

    public int getPoints() {
        return points;
    }

    public List<String> getAllAnswers() {
        List<String> allAnswers = new ArrayList<>(incorrectAnswers);
        allAnswers.addAll(correctAnswers);
        Collections.shuffle(allAnswers);
        return allAnswers;
    }
}

```

The following code is responsible for declaring the Question class, used for instantiating question objects, which are created by deserializing YAML files using the Jackson library. Upon initializing an ObjectMapper, used to map the text in the file and determine the adequate object from it, a try-with-resources statement is used to ensure exception handling, but also to ensure that the resources created will be closed upon exiting the block of code. This block of code creates initializes an InputStream for the 'questions.yml' file, as well as a MappingIterator, both used to read all of the question objects inside of the file. After being mapped to the Question class, all of the objects are read and assigned to the questions list. A big advantage of this approach is that it allows for the simple creation of new questions via a YML file, and is a safe and simple approach to deserialization. However, the negatives of this approach is that a small mistake in the YAML formatting will cause exceptions, and the exception handling, whilst safe,

provides little to no useful information to the program user, as they might not know what caused the issue.

Once again, the testing strategy is effective in testing the above approach in multiple different tests, all that refer to testing question answering, writing and saving, as testing them with normal, abnormal and extreme data will make sure that the above technique is well-tested and safe.

Technique 4: Singleton

```
class QuestionManager {
    private static QuestionManager INSTANCE;
    private final List<Question> questions;
    private Iterator<Question> questionIterator;
    private int totalPoints;

    public static QuestionManager getInstance() {
        return Objects.requireNonNullElseGet(INSTANCE, QuestionManager::new);
    }

    private QuestionManager() {
        //further code...
        INSTANCE = this;
        //further code...
    }

    //further code here not relevant to the definition of the technique
}
```

A singleton is a class which contains only 1 instance. The QuestionManager class is an example of such, this is because it contains sensitive data that should be encapsulated, but still not static, and persisting to one object. To achieve this, the constructor of the class is private and inaccessible to other classes, and is instantiated only once, in the getInstance() method, which returns the only instance/object of the class if it is not null, or initializes it if it is. The major advantage of this approach is that it encapsulates the questions and points variables, which are fields common anywhere in the program, but should be protected from unsafe access and should not be static fields.

Technique 5: Action Listeners with Lambda Expressions

```
startButton.addActionListener(e -> {
    QuestionManager questionManager = QuestionManager.getInstance();
    Question question = questionManager.getNextQuestion();
    new QuestionMenu(question).open();
    dispose();
});
```

The code above states what the start button of the main menu should do once clicked. The Java Swing 'addActionListener' method provides a click action for all buttons, in order to state what this action should be, a functional interface (an interface containing only one function) called ActionListener is passed as a parameter, the value of its parameter function will be run once the button is clicked. Instead of initializing the entire interface, we can write a lambda expression which's code block will be run upon clicking the button. Here we put our code block initializing our question menu window. The advantage of using lambda expressions is that it reduces boilerplate code and provides a simple way to use functional interfaces, but its limitation is that they are hard to debug and their execution can be slow and damage performance.

Criterion E: Evaluation

Evaluating the success criteria

Success Criterion 1: Using an external 'questions.yml' file, the user of the product can devise as many multiple-choice questions as they wish with an arbitrary amount of incorrect or correct answers. This criterion has been met.

Success Criterion 2: Using the beforementioned file, a user can state the URL of the desired image and the app is able to load it from the internet if internet connection is available. If no URL has been stated, the snippet of code loading the image will be skipped, requiring no internet connection. This criterion has been met, however, I should have implemented it so that the application would not produce an exception if the URL is invalid or there is no internet connection.

Success Criterion 3: Every question must have a set amount of points dedicated to it, set via the same YML file. This criterion is met. However, the values also allow negative numbers, which would actually deduce the amount of points for a correct answer, which doesn't really fit the meaning of the product, however, doesn't actually go against the success criteria.

Success Criterion 4: Using the 'savedata.txt' file, the product successfully stores the obtained points persistently. This criterion is met, however, nothing stops the user from accessing and modifying this file and therefore cheating, which is a limitation of the approach.

Success Criterion 5: Every single time a question is displayed, the order of all of the possible correct and incorrect answers is randomized/shuffled. This criterion is met.

Success Criterion 6: When the answer to a question has been selected, the application displays a pop-up stating whether the answer was correct, and if so, how many points were obtained. Additionally, regardless of the answer, an explanation to the answer of the question is displayed, which is set in the 'questions.yml' file. This criterion has been met.

Improvement justification

Whilst the app meets all of the success criteria, there are some improvements that can be added for further functionality and extensibility. This includes a more visually appealing user interface, in order to attract users and develop a more immersive experience. Another improvement could be a more simple way to introduce new questions, such as adding a menu to create a question, instead of managing everything via a YML file, which would enhance user experience and make the app simpler to use for the less-experienced. Also, I would implement it so that more than one answer can be selected, since the app currently only allows the user to select one answer, and checks its correctness in the set of all correct answers, so implementing bonus points for giving multiple correct answers would surely improve the immersion and rewarding of knowledge.